

AUTOMATED PODCAST TRANSCRIPTION & TOPIC SEGMENTATION.

1. Title

Podcast Intelligence Dashboard.

2. Overview

Podcasts often span several hours, making it difficult to locate specific discussions or topics. This system solves that problem by:

Converting audio into accurate text transcripts.

Structuring transcripts into smaller semantic segments.

Extracting representative keywords for each segment.

Generating initial summaries for quick understanding.

3. Objectives

Transcribe podcast audio using fast and accurate speech-to-text models.

Preprocess and normalize audio for better transcription quality.

Segment transcripts into manageable, topic-oriented chunks.

Extract keywords for each segment to support topic discovery.

Generate initial summaries to provide high-level context.

Maintain a clean, modular, and extensible pipeline.

4. Approach

The project follows a **modular, pipeline-based approach** to transform raw podcast audio into structured and analyzable information. The process begins with audio ingestion and preprocessing, where user-uploaded audio files are normalized to ensure stable downstream processing. Preprocessing includes silence removal, noise reduction, mono-channel conversion, and resampling to 16 kHz using FFmpeg-based utilities. This step reduces unnecessary computation and ensures compatibility with the speech recognition model.

For transcription, the system employs **OpenAI's Whisper**, a transformer-based speech-to-text model known for its robustness to accents and audio variations, making it well-suited for the TED Talks dataset. Whisper converts the preprocessed audio into a detailed text transcript by first generating log-Mel spectrograms and then using an encoder–decoder architecture to predict text tokens with proper punctuation. This produces a high-quality transcript that serves as the foundation for further analysis.

Once transcription is complete, the text is **sequentially segmented** into non-overlapping topic-based sections to preserve the natural flow of the audio. Each segment undergoes lightweight NLP analysis, including keyword extraction through frequency-based filtering and sentiment analysis using TextBlob. The results are presented through an interactive Streamlit dashboard that enables users to explore transcripts, topics, sentiment, and keyword trends efficiently, eliminating the need to listen to the entire audio recording.

5. Setup

This section describes the steps required to configure and run the project locally.

Prerequisites

Before setting up the project, ensure the following software is installed on your system:

- **Python 3.9 or higher**

- **FFmpeg**, installed and added to the system PATH
- **Git**, for version control

FFmpeg is required for audio preprocessing, while Python is used for executing the transcription and analysis pipeline.

Installation Steps

1. Clone the Repository

```
git clone https://github.com/springboardmentor13579x-proj/Automated-Podcast-Transcription-and-Topic-Segmentation.git
```

```
cd Automated-Podcast-Transcription-and-Topic-Segmentation
```

2. Create and Activate a Virtual Environment

Windows

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

```
source .venv/bin/activate
```

Using a virtual environment ensures dependency isolation and prevents conflicts with other Python projects.

3. Install Project Dependencies

```
pip install -r requirements.txt
```

This installs all required libraries, including Whisper, Streamlit, Plotly, TextBlob, and audio processing dependencies.

Verification

After installation, verify the setup by running:

```
streamlit run app.py
```

If the Streamlit interface opens in the browser (typically at <http://localhost:8501>), the setup is complete and the application is ready for use.

6. Usage

Place TED Talk or podcast audio files (.wav, .mp3, .m4a) in the audio_raw/ directory or upload them through the Streamlit interface.

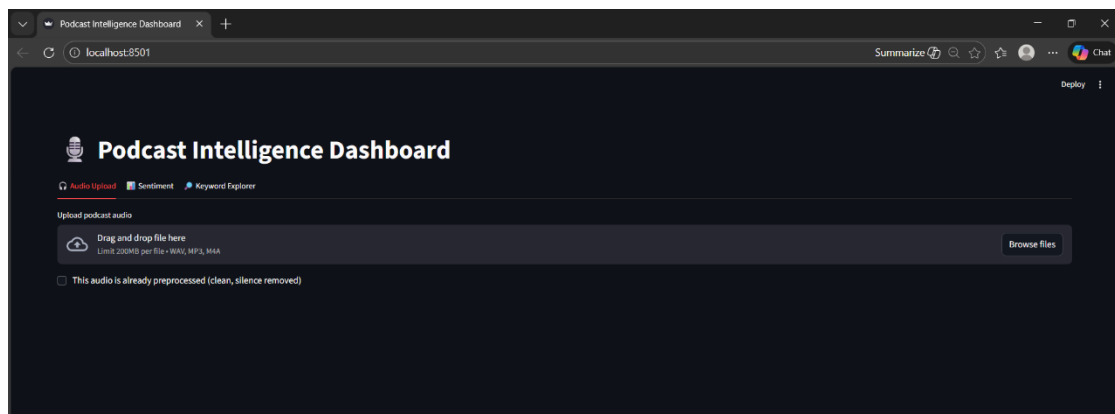
Run the backend pipeline or Streamlit app to preprocess audio, generate transcripts, and segment long talks into topic-based sections.

Explore each segment using extracted keywords to quickly locate relevant parts of long TED Talks or lectures.

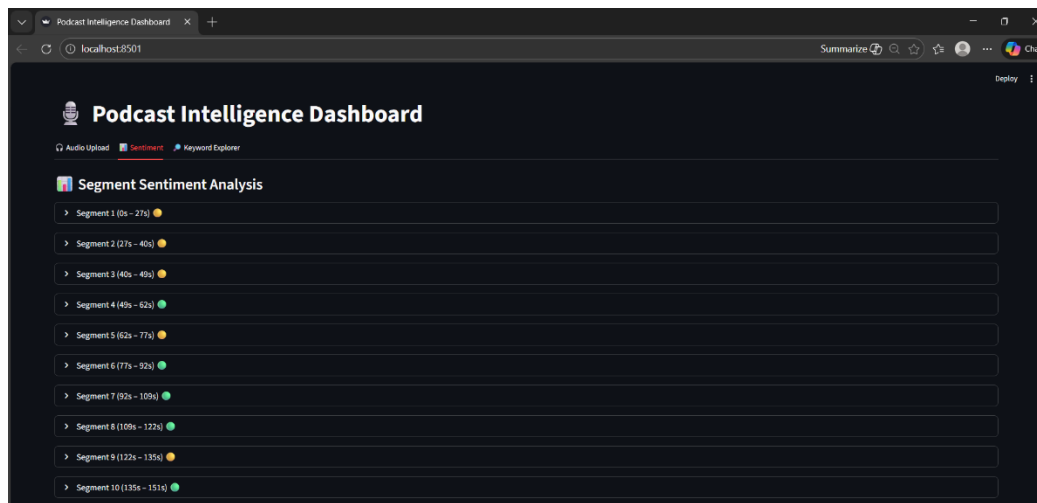
View initial summaries and sentiment insights to gain a high-level understanding without reading the full transcript.

Use the dashboard for interactive analysis and exploration rather than real-time processing.

App Landing Page:



Sentiment Analysis:



7. Troubleshooting

FFmpeg Not Found:

Ensure FFmpeg is installed and added to the system PATH; restart the terminal after installation.

Streamlit Command Not Recognized:

Activate the virtual environment before running the application.

Transcription Crashes or Tensor Errors:

This may occur if the audio becomes empty after preprocessing; try skipping preprocessing or using a cleaner audio file.

Out of Memory During Transcription:

Reduce the Whisper model size (tiny or base) when running on CPU-only systems.

8. Limitations & Challenges

Transcription accuracy depends heavily on audio quality; noisy or accented speech may reduce performance.

Large Whisper models require significant memory, which can be challenging on low-resource systems.

Topic segmentation is heuristic-based and may not always perfectly align with true conversational shifts.

Keyword extraction and summarization rely on statistical methods and may miss deeper semantic meaning.

The system is optimized for offline analysis of datasets like TED Talks, not large-scale real-time deployment.

9. Future Scope

Use faster transcription frameworks or GPU acceleration to reduce processing time for long audio files.

Add automatic summarization using large language models to generate concise episode or segment summaries.

Extend the system to support real-time or live audio transcription.

Improve topic segmentation using semantic similarity or transformer-based topic modeling techniques.

Deploy the application on cloud platforms to enable public and scalable access.

10. Reference

1. OpenAI Whisper

<https://github.com/openai/whisper>

2. Streamlit (Official Documentation)

<https://docs.streamlit.io/>

3. Scikit-learn (Machine Learning Library Documentation)

<https://scikit-learn.org/>

4. Plotly Express (Visualization Library)

<https://plotly.com/python/plotly-express/>

5. TextBlob (Sentiment Analysis Library)

<https://textblob.readthedocs.io/>